Matière : **Conception et développement d'applications**
Thème : **Préparation épreuve E6 BTS SIO option SLAM**
Application 1 : **Cahier des charges techniques**
Projet JO 2028 - Valentin GALAIS - Version 1.0

Sommaire

1. Contexte du projet....2
1.1. Présentation du projet....2
1.2. Date de rendu du projet....2
2. Besoins fonctionnels....2
3. Ressources nécessaires à la réalisation du projet....3
3.1. Ressources matérielles....3
3.2. Ressources logicielles....3
4. Gestion du projet....4
5. Conception du projet....4
5.1. Arborescence....4
5.2. Wireframes....5
5.3. Maquettes....5
5.4. Le back-end....7
5.4.1. Diagramme de cas d'utilisation....7
5.4.2. Diagramme d'activités....7
5.4.3. Modèle Conceptuel de Données (MCD)....8
5.4.4. Modèle Logique de Données (MLD)....8
5.4.5. Modèle Physique de Données (MPD)....9
6. Technologies utilisées....9
6.1. Langages de développement Web....9
6.2. Base de données....10
7. Sécurité....10
7.1. Login et protection des pages administrateurs....10
7.2. Hachage des mots de passe avec Bcrypt....11
7.3. Protection contre les attaques XSS (Cross-Site Scripting)....12
7.4. Protection contre les injections SQL....12

1. Contexte du projet

1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

Les Jeux Olympiques d'été se dérouleront à Los Angeles (États-Unis) du 14 au 30 juillet 2028. Le présent cahier des charges techniques décrit la solution retenue : les fonctionnalités attendues, les ressources et technologies mobilisées, la conception (maquettes, diagrammes UML, modèles de données) et les mesures de sécurité mises en œuvre. Il s'appuie sur les livrables de la mission 2 (wireframes, maquettes, arborescence), sur les modèles de données et diagrammes UML fournis, et sur la trame d'application PHP.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard **début novembre 2026**.

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

Partie publique (accessible à tout usager, sans authentification) :

Réf.	Fonctionnalité	Description
F-01	Accueil	Trois accès : Sports, Calendrier des épreuves, Résultats, et menu de navigation commun à toutes les pages.
F-02	Liste des sports	Affiche tous les sports par ordre alphabétique.
F-03	Calendrier des épreuves	Affiche la date, l'heure, le sport et le lieu de chaque épreuve, en ordre chronologique.
F-04	Liste des résultats	Affiche le sport, le nom et le prénom de l'athlète, son pays et son résultat.
F-05	Accès administrateur	Lien vers le formulaire de connexion (login et mot de passe).

Partie privée (réservée aux administrateurs authentifiés) :

Réf.	Module	Opérations	Règles de gestion principales
A-01	Administrateurs	Lister, ajouter, modifier, supprimer	Login unique ; mot de passe haché ; un administrateur ne peut pas supprimer son propre compte.
A-02	Sports	Lister, ajouter, modifier, supprimer	Nom obligatoire et unique ; suppression refusée si le sport est lié à une épreuve.
A-03	Lieux	Lister, ajouter, modifier, supprimer	Nom, adresse, code postal, ville ; suppression refusée si le lieu est lié à une épreuve.

Réf.	Module	Opérations	Règles de gestion principales
A-04	Pays	Lister, ajouter, modifier, supprimer	Nom unique ; suppression refusée si le pays est lié à un athlète.
A-05	Genres	Lister, ajouter, modifier, supprimer	Nom unique ; suppression refusée si le genre est lié à un athlète.
A-06	Athlètes	Lister, ajouter, modifier, supprimer	Nom, prénom, pays et genre obligatoires.
A-07	Calendrier des épreuves	Lister, ajouter, modifier, supprimer	Date, heure, lieu et sport obligatoires.
A-08	Résultats	Lister, ajouter, modifier, supprimer	Un résultat associe un athlète à une épreuve (couple unique) ; valeur en texte libre (temps, distance, hauteur, mention).

Règles transversales : confirmation avant toute suppression, message de succès ou d'erreur après chaque action, validation systématique des données côté serveur, affichage adapté à l'ordinateur, à la tablette et au smartphone (responsive).

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

Ressource	Usage
Ordinateur de développement (PC ou Mac, 8 Go de RAM minimum)	Écriture du code, serveur local (Apache, PHP, MySQL), conception des maquettes et des schémas.
Smartphone et tablette (ou mode appareil du navigateur)	Tests de l'affichage responsive.
Connexion Internet	Travail collaboratif sur GitHub, Trello et Figma ; téléchargement des outils.
Serveur d'hébergement (à définir avec le commanditaire)	Mise en ligne finale : serveur web Apache avec PHP 8 et MySQL.

3.2. Ressources logicielles

Logiciel	Usage
Visual Studio Code	Éditeur de code (PHP, HTML, CSS, SQL).
WAMP / MAMP / XAMPP	Serveur local Apache + PHP + MySQL.
phpMyAdmin	Administration de la base de données, import des scripts SQL.
Git et GitHub	Gestion de versions et dépôt du projet (jo2028-galais-valentin).
Trello	Gestion de projet en méthode Kanban.
Figma	Conception des wireframes et des maquettes.
Visual Paradigm	Diagrammes UML (cas d'utilisation, activités) et plan du site.
Mocodo	Modèles Merise (MCD, MLD, MPD).
Navigateurs (Chrome, Firefox, Edge, Safari)	Tests de compatibilité et outils de développement.

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.

Le tableau Trello est organisé en colonnes qui représentent l'avancement de chaque tâche, chaque tâche étant une carte déplacée de gauche à droite :

Colonne	Rôle	Exemples de cartes
À faire	Tâches identifiées et priorisées.	Pages publiques ; style responsive ; modules admin (lieux, pays, genres, athlètes, épreuves, résultats) ; tests et recette.
En cours	Tâches en développement (2 cartes maximum par personne).	Rédaction du cahier des charges ; dépôt GitHub ; authentification et sessions.
En test	Fonctionnalités terminées, en vérification.	Page d'accueil et menu ; module admin Sports.
Terminé	Tâches validées.	Maquettes Figma ; arborescence ; diagrammes UML ; MCD, MLD, MPD ; base de données et jeu de test.

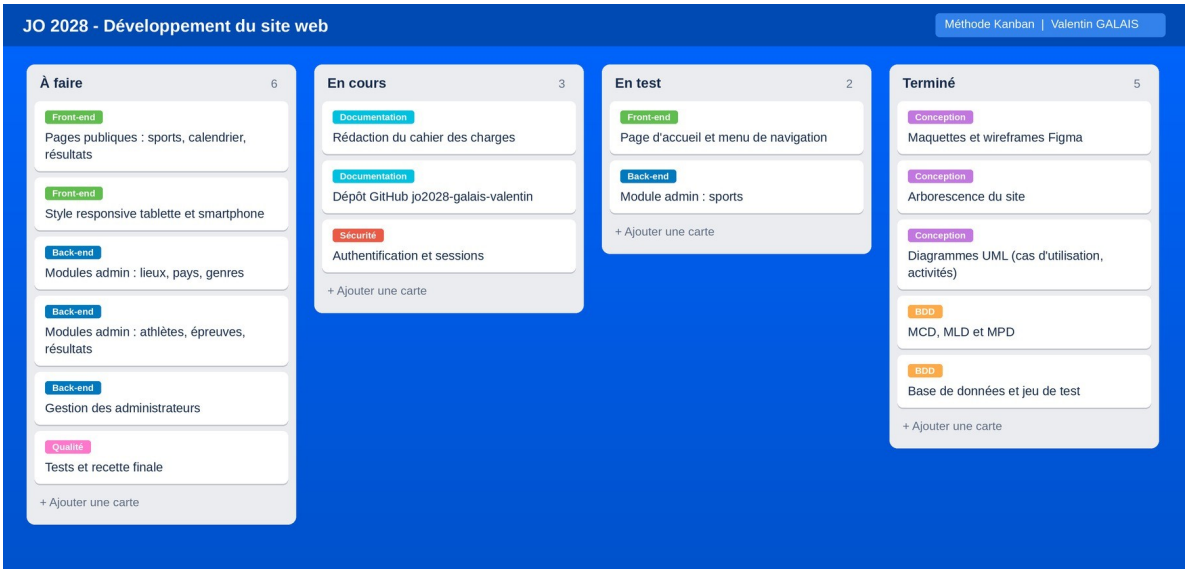


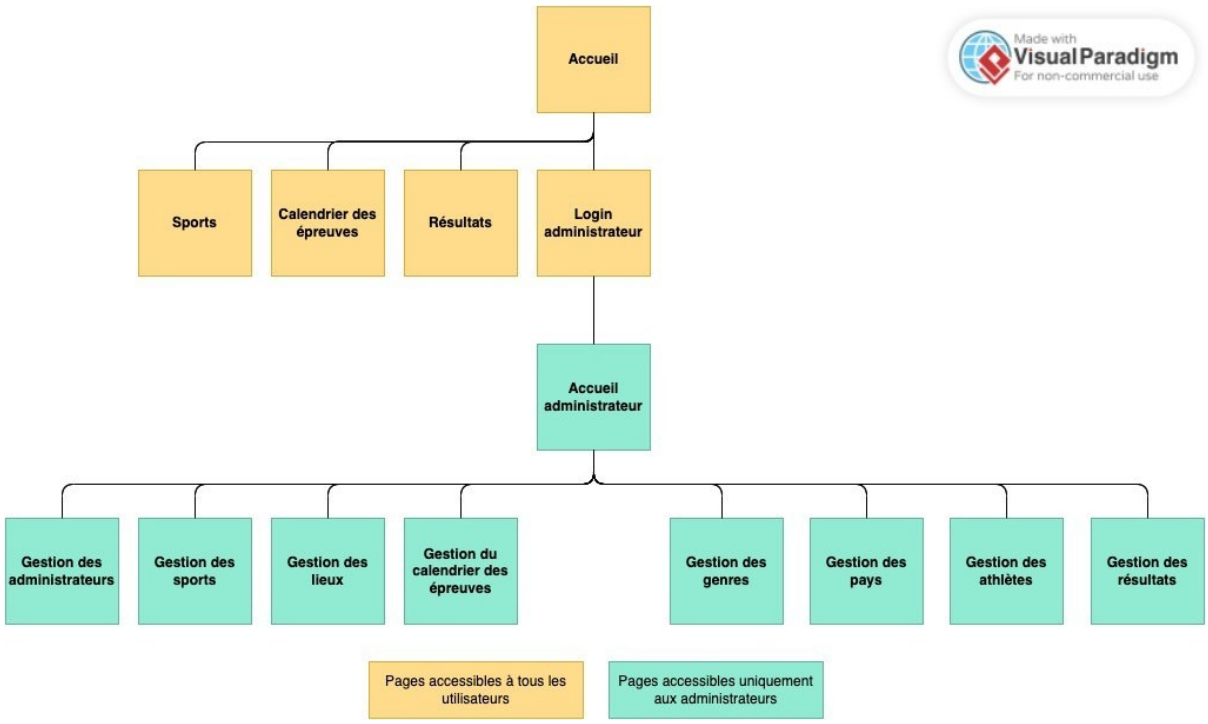
Tableau Kanban du projet JO 2028 (organisation sous Trello)

Nous travaillons également sur GitHub, plateforme de développement collaboratif. Le dépôt **jo2028-galais-valentin** contient le cahier des charges au format PDF puis le code source de l'application. Les modifications sont enregistrées par des commits fréquents aux messages explicites (par exemple « Ajout du module de gestion des lieux »), la branche principale `main` restant toujours fonctionnelle.

5. Conception du projet

5.1. Arborescence

Le plan du site distingue les pages accessibles à tous les utilisateurs (en jaune) des pages réservées aux administrateurs (en vert).



Arborescence du site JO 2028

Page	Fichier de l'application	Accès
Accueil	index.php	Public
Sports	pages/sports.php	Public
Calendrier des épreuves	pages/events.php	Public
Résultats	pages/results.php	Public
Login administrateur	pages/login.php (traitement : database/auth.php)	Public
Accueil administrateur	pages/admin/admin.php	Administrateur
Gestion des administrateurs, sports, lieux, pays, genres, athlètes, épreuves, résultats	pages/admin/admin-xxx/ : manage, add, modify, delete	Administrateur

5.2. Wireframes

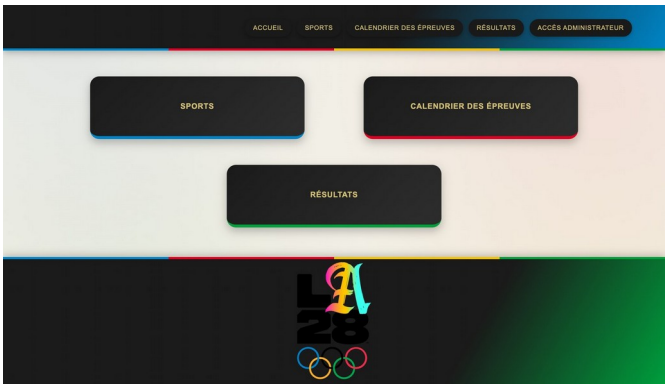
Les wireframes et les maquettes ont été réalisés dans Figma, dans le projet « wireframe JO 2028 » :

https://www.figma.com/design/t7NBJLHyl9TgSPWx45cOoI/wireframe-JO-2028?m=auto&t=6DUENau1wLPFmoyh-6

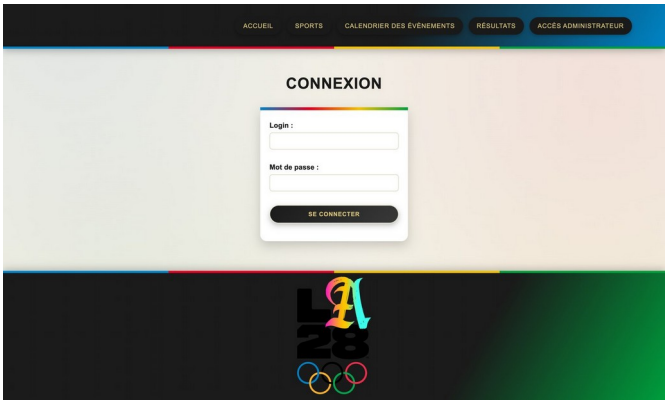
Toutes les pages partagent la même ossature : un en-tête sombre contenant le menu (Accueil, Sports, Calendrier des épreuves, Résultats, Accès administrateur), une zone de contenu centrale, puis un pied de page affichant le logo LA28. Trois gabarits sont déclinés : l'accueil (trois cartes d'accès), la connexion (formulaire centré) et les tableaux de données (sports, calendrier, résultats).

5.3. Maquettes

Les maquettes ci-dessous sont déclinées pour trois formats d'écran : ordinateur, tablette et smartphone. Elles suivent la charte olympique : anneaux bleu, rouge, jaune, vert et noir, fond clair, en-tête et pied de page sombres, boutons arrondis et typographie sans empattement.



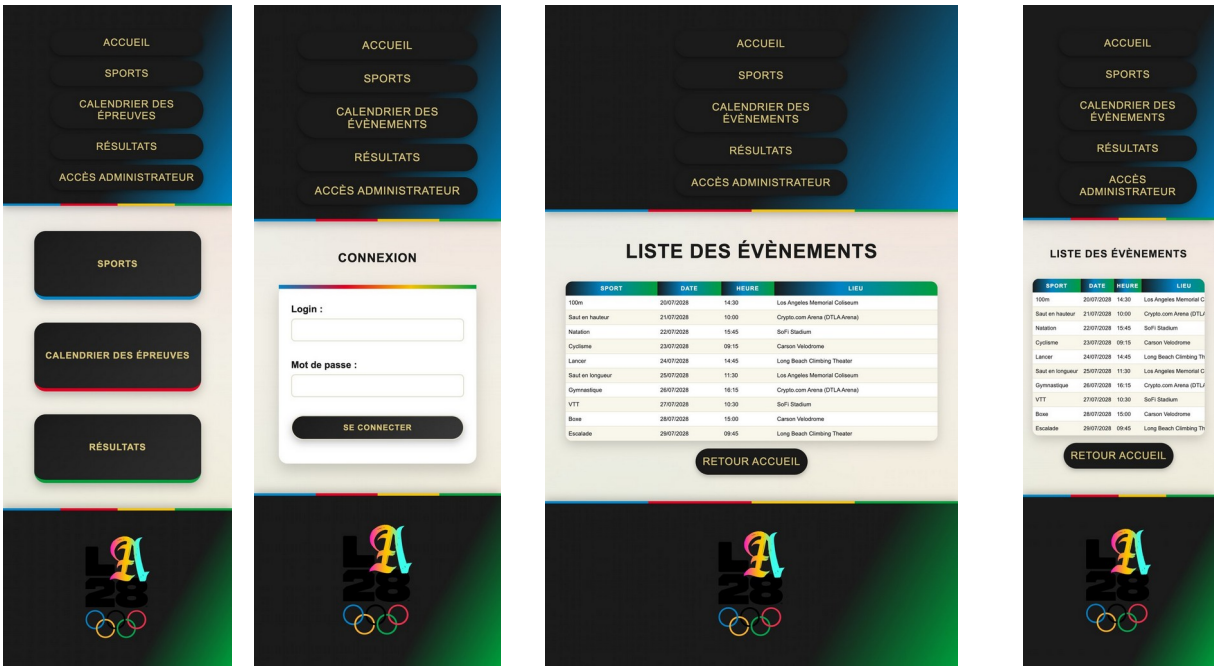
Maquette de la page d'accueil (ordinateur)



Maquette de la page de connexion (ordinateur)



Maquette du tableau des résultats (ordinateur), identique pour les sports et le calendrier



Maquettes responsive : accueil, connexion, tableau (tablette), tableau (smartphone)

5.4. Le back-end

5.4.1. Diagramme de cas d'utilisation

L'**usager** consulte les informations (sports, calendrier des épreuves, résultats). L'**administrateur** hérite de ces droits et, après connexion, gère l'ensemble des informations : administrateurs, sports, lieux, calendrier des épreuves, pays, athlètes et résultats.

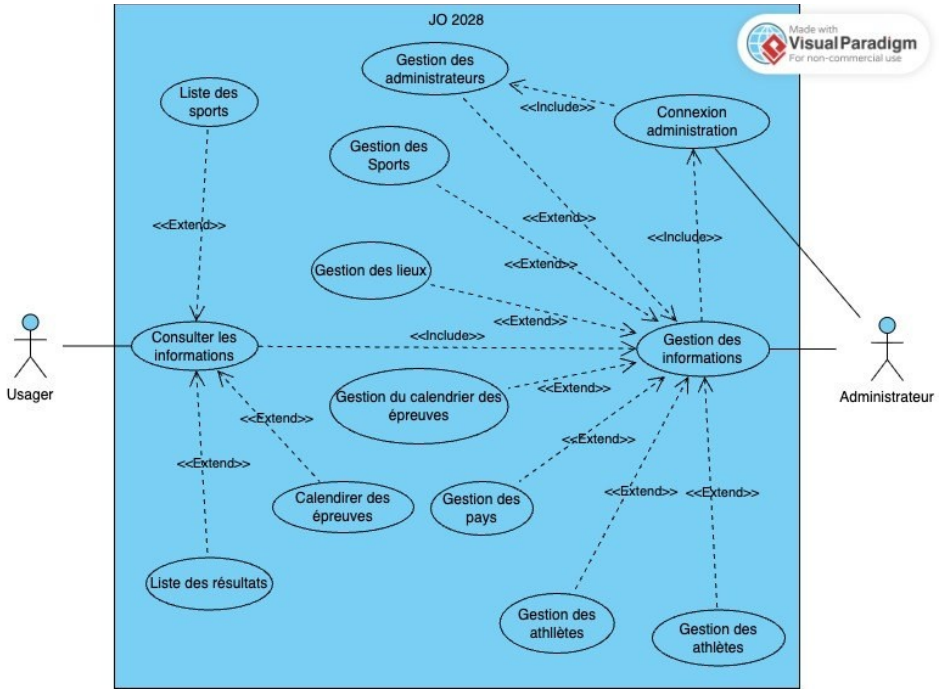


Diagramme de cas d'utilisation

5.4.2. Diagramme d'activités

Le diagramme décrit l'authentification : l'administrateur saisit son login et son mot de passe, le système les vérifie, puis donne accès à l'espace administrateur si les données sont correctes. Sinon, une erreur est affichée et le formulaire est proposé de nouveau.

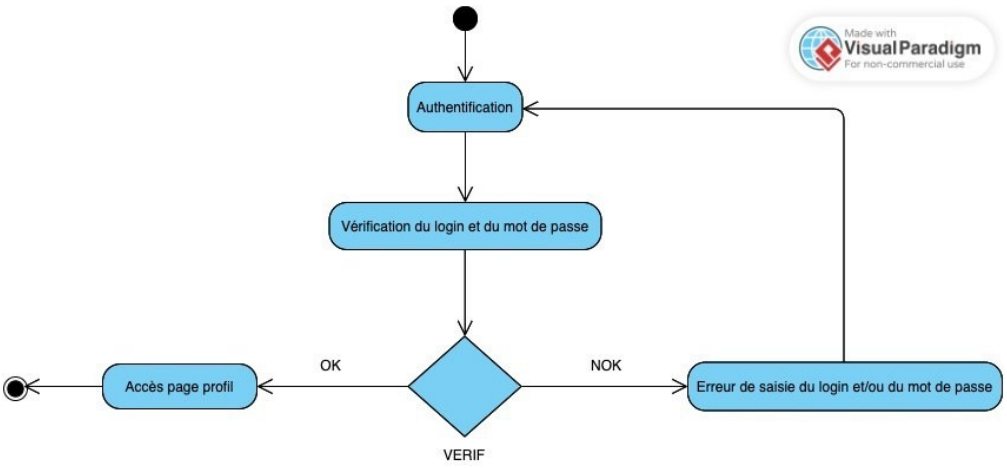
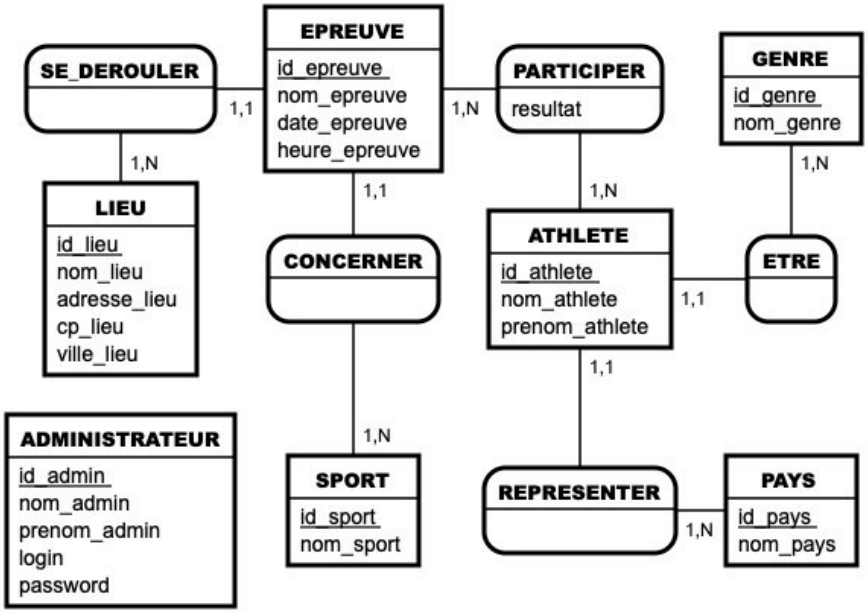


Diagramme d'activités : connexion de l'administrateur

5.4.3. Modèle Conceptuel de Données (MCD)

Un **athlète** représente un pays, a un genre et participe à des épreuves (avec un résultat). Une **épreuve** concerne un sport et se déroule dans un lieu. La table **administrateur** est indépendante des autres.



Modèle Conceptuel de Données (Merise)

5.4.4. Modèle Logique de Données (MLD)

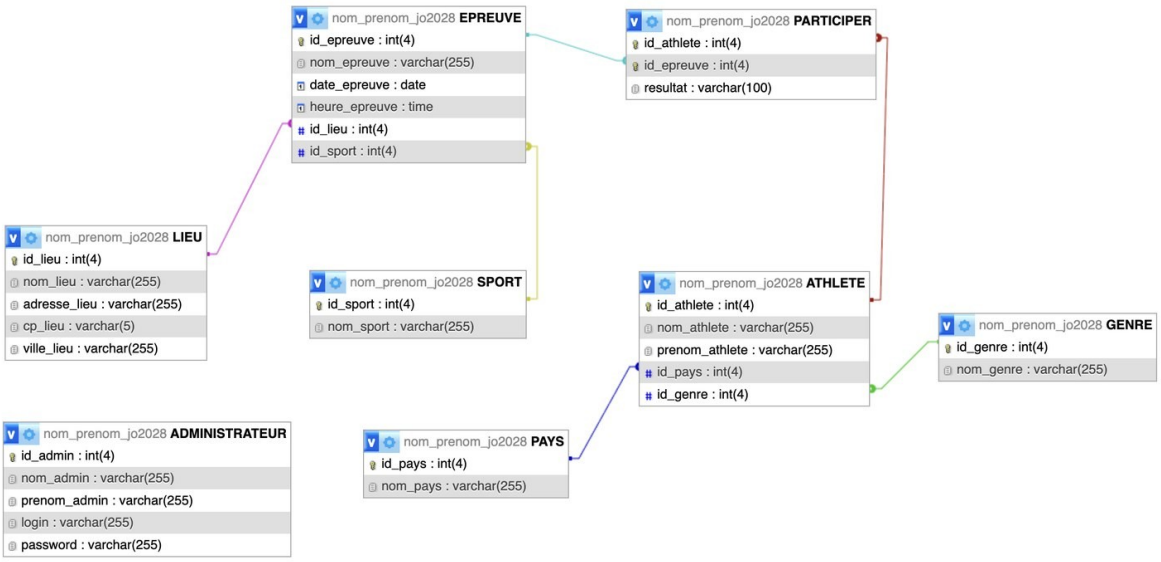
Les clés primaires sont soulignées et les clés étrangères précédées de #.

Relation	Attributs
ATHLETE	id_athlete, nom_athlete, prenom_athlete, #id_pays, #id_genre
EPREUVE	id_epreuve, date_epreuve, heure_epreuve, #id_lieu, #id_sport
PARTICIPER	#id_athlete, #id_epreuve, resultat (clé primaire composée de #id_athlete et #id_epreuve)
GENRE	id_genre, nom_genre
LIEU	id_lieu, nom_lieu, adresse_lieu, cp_lieu, ville_lieu
PAYS	id_pays, nom_pays
SPORT	id_sport, nom_sport
ADMINISTRATEUR	id_admin, nom_admin, prenom_admin, login, password

Remarque : ce MLD est aligné sur le MPD et sur le script `db-jo.sql` : l'association est implémentée par la table PARTICIPER (attribut `resultat`) et l'attribut `nom_epreuve`, présent sur le MCD, n'est pas créé dans le script SQL.

5.4.5. Modèle Physique de Données (MPD)

Le MPD correspond au script `db-jo.sql` (moteur InnoDB, clés étrangères déclarées avec `ALTER TABLE ... ADD FOREIGN KEY`). Le script `test-data-jo.sql` fournit un jeu d'essai : 10 pays, 2 genres, 5 lieux, 10 sports, 10 athlètes, 10 épreuves, leurs résultats et un compte administrateur.



Modèle Physique de Données

Table	Champs et types	Contraintes
ATHLETE	id_athlete INT(4) ; nom_athlete, prenom_athlete VARCHAR(255) ; id_pays, id_genre INT(4)	PK id_athlete (auto-incrément) ; FK id_pays, id_genre NOT NULL
EPREUVE	id_epreuve INT(4) ; date_epreuve DATE ; heure_epreuve TIME ; id_lieu, id_sport INT(4)	PK id_epreuve ; FK id_lieu, id_sport NOT NULL
PARTICIPER	id_athlete, id_epreuve INT(4) ; resultat VARCHAR(100)	PK (id_athlete, id_epreuve) ; FK vers ATHLETE et EPREUVE
GENRE / PAYS / SPORT	id INT(4) ; nom VARCHAR(255)	PK auto-incrément
LIEU	id_lieu INT(4) ; nom_lieu, adresse_lieu, ville_lieu VARCHAR(255) ; cp_lieu VARCHAR(5)	PK id_lieu
ADMINISTRATEUR	id_admin INT(4) ; nom_admin, prenom_admin, login, password VARCHAR(255)	PK id_admin ; password = hash Bcrypt

6. Technologies utilisées

6.1. Langages de développement Web

Langage	Rôle dans le projet
HTML5	Structure des pages avec des balises sémantiques (header, nav, main, footer) et les formulaires.

Langage	Rôle dans le projet
CSS3	Mise en forme selon les maquettes : `styles-computer.css` pour l'affichage de base, `styles-responsive.css` (media queries) pour tablette et smartphone, `normalize.css` pour harmoniser les navigateurs.
PHP 8	Langage côté serveur : traitement des formulaires, sessions, authentification, génération des pages et accès à la base via PDO.
SQL	Création de la base et requêtes (SELECT avec jointures, INSERT, UPDATE, DELETE).

L'application est organisée en pages PHP, avec la structure de dossiers suivante :

```
nom-prenom-app-jo2028/
  index.php                 accueil public
  css/                      normalize.css, styles-computer.css, styles-responsive.css
  img/                      logo-jo.png, favicon.ico
  database/                 database.php (connexion PDO), auth.php (connexion admin)
  pages/                    sports.php, events.php, results.php, login.php, logout.php
  pages/admin/              admin.php + un dossier par module de gestion
                            (admin-sports, admin-places, admin-countries, admin-genders,
                             admin-athletes, admin-events, admin-results, admin-users)
```

6.2. Base de données

La base de données est **MySQL** (ou MariaDB), gérée avec phpMyAdmin. Les tables utilisent le moteur **InnoDB**, qui permet de déclarer les clés étrangères et donc de garantir l'intégrité des données (un sport lié à une épreuve ne peut pas être supprimé sans précaution). La connexion depuis PHP se fait avec **PDO**, en mode exceptions, ce qui permet d'utiliser des requêtes préparées et de gérer proprement les erreurs :

```
$dsn = "mysql:host=$host;dbname=$dbname;charset=utf8";
$connexion = new PDO($dsn, $username, $password);
$connexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

7. Sécurité

7.1. Login et protection des pages administrateurs

L'authentification repose sur les **sessions PHP**. Après la saisie du formulaire, `auth.php` recherche l'administrateur correspondant au login (requête préparée) puis vérifie le mot de passe avec `password_verify()`. Si les informations sont correctes, l'identité de l'administrateur est enregistrée dans la session et il est redirigé vers l'accueil d'administration. Sinon, un message d'erreur volontairement générique (« Login ou mot de passe incorrect. ») est affiché, sans indiquer lequel des deux champs est faux.

```
if ($row && password_verify($password, $row["password"])) {
    session_regenerate_id(true);               // évite la fixation de session
    $_SESSION["id_admin"]     = $row["id_admin"];
    $_SESSION["nom_admin"]    = $row["nom_admin"];
    $_SESSION["prenom_admin"] = $row["prenom_admin"];
    $_SESSION["login"]        = $row["login"];
    header("location: ../pages/admin/admin.php");
    exit();
}
```

Protection des pages administrateurs. Chaque page d'administration commence par démarrer la session et contrôler qu'un administrateur est connecté. À défaut, l'internaute est redirigé vers l'accueil public et le script est arrêté par `exit()`, de sorte qu'une page privée ne peut pas être consultée en saisissant directement son adresse.

```
session_start();
if (!isset($_SESSION['login'])) {
    header('Location: ../../index.php');
    exit();
}
```

Protection CSRF. Un jeton aléatoire est généré à l'ouverture de la session, placé dans un champ caché de chaque formulaire, puis comparé à celui de la session lors du traitement avec `hash_equals()`. Une requête provenant d'un autre site est ainsi rejetée. La déconnexion (`logout.php`) vide puis détruit la session.

```
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
// dans le formulaire :
<input type="hidden" name="csrf_token"
       value="<?= $_SESSION['csrf_token'] ?>">
// au traitement :
if (!isset($_POST['csrf_token']) ||
    !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
    $_SESSION['error'] = "Token CSRF invalide.";
    exit();
}
```

7.2. Hachage des mots de passe avec Bcrypt

Bcrypt est un algorithme de hachage conçu pour les mots de passe. Il transforme le mot de passe en une empreinte de 60 caractères, impossible à inverser, à laquelle il ajoute un **sel** aléatoire (deux mots de passe identiques donnent deux empreintes différentes) et dont le coût de calcul est volontairement élevé et réglable, ce qui ralentit les attaques par force brute. Ainsi, même si la base de données est dérobée, les mots de passe ne sont pas lisibles : ils ne sont **jamais stockés en clair**.

En PHP, `password_hash()` avec `PASSWORD_BCRYPT` crée l'empreinte à l'enregistrement d'un administrateur, et `password_verify()` la compare au mot de passe saisi lors de la connexion. Le champ `password` de la table ADMINISTRATEUR (VARCHAR(255)) est assez grand pour stocker l'empreinte.

```
// Création / modification d'un administrateur
$hash = password_hash($motDePasse, PASSWORD_BCRYPT);
// $hash ressemble à : $2y$10$N9qo8uLOickgx2ZMRZoMye...  (60 caractères)
$sql = "INSERT INTO ADMINISTRATEUR (nom_admin, prenom_admin, login, password)
        VALUES (:nom, :prenom, :login, :password)";
$stmt = $connexion->prepare($sql);
$stmt->bindParam(":password", $hash, PDO::PARAM_STR);

// Connexion
if (password_verify($motDePasseSaisi, $row['password'])) { /* accès autorisé */ }
```

7.3. Protection contre les attaques XSS (Cross-Site Scripting)

Une attaque **XSS** consiste à injecter du code (généralement JavaScript) dans une page web, par exemple via un champ de formulaire. Si ce contenu est réaffiché tel quel, le navigateur des visiteurs l'exécute : vol de session, redirection vers un faux site, modification de la page, etc.

En PHP, la protection consiste à ne jamais afficher une donnée sans la neutraliser. Les caractères spéciaux (<, >, &, guillemets) sont convertis en entités HTML avec `htmlspecialchars()` au moment de l'affichage, et les entrées sont filtrées avec `filter_input()` à la réception. Un texte comme `<script>alert(1)</script>` est alors affiché comme du texte et n'est pas exécuté.

```
// À la réception
$nomSport = filter_input(INPUT_POST, 'nomSport', FILTER_SANITIZE_SPECIAL_CHARS);

// À l'affichage
<td><?= htmlspecialchars($row['nom_sport'], ENT_QUOTES, 'UTF-8') ?></td>
```

7.4. Protection contre les injections SQL

Une **injection SQL** consiste à saisir dans un champ du code SQL qui modifie la requête exécutée par le serveur. Par exemple, avec une requête construite par concaténation, la saisie `' OR '1'='1` dans le champ login permettrait de contourner l'authentification, et des commandes comme `DROP TABLE` pourraient détruire des données.

En PHP, la solution est d'utiliser des **requêtes préparées PDO** : la requête est envoyée au serveur avec des paramètres nommés (`:login`), puis les valeurs sont liées séparément avec `bindParam()`. Le contenu saisi est alors traité comme une simple valeur et jamais comme du code SQL. Aucune variable n'est concaténée dans une requête.

```
// À ne pas faire :
// $query = "SELECT * FROM ADMINISTRATEUR WHERE login = '$login'";

// Requête préparée :
$query = "SELECT id_admin, nom_admin, prenom_admin, login, password
          FROM ADMINISTRATEUR WHERE login = :login";
$stmt = $connexion->prepare($query);
$stmt->bindParam(":login", $login, PDO::PARAM_STR);
$stmt->execute();
```

Ces protections sont complétées en production par le passage en HTTPS et la désactivation de l'affichage des erreurs PHP (`display_errors = 0`) pour ne pas révéler d'informations sur la structure de l'application.